Advanced Rust 2026 - Lab 4: Async Rate Limiter Library

Lukáš Hozda

Spring 2026

Introduction

In this lab, you will build a small async Rust library rather than a one-off program. The point is not only to make it work, but to make the public API tolerable to use, test, and extend.

The concrete task is an async rate limiter. This is a realistic library-shaped problem: there is shared state, concurrent access, waiting, time-based logic, and a small registry layer on top.

Setup

1. Create a new library project: `cargo new --lib async_rate_limiter`
2. Add Tokio to your dependencies. A minimal setup is enough:

```
[dependencies]
tokio = { version = "1", features = ["macros", "rt-multi-thread", "time", "test-util"] }
```

You may add further dependencies if you really need them, but this task does not require much beyond Tokio and the standard library.

Exercise: Implement an Async Rate Limiter Library

Objective

Implement a thread-safe async rate limiter library with:

1. a token-bucket limiter
2. a small registry for named limiters
3. tests and rustdoc comments for the public API

The goal is deliberately library-oriented rather than application-oriented.

Required API

```rust
use std::sync::Arc;
use std::time::Duration;

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Error {
    Timeout,
    InvalidConfig,
}

pub struct RateLimiter {
    // implementation details hidden
```

```rust
}

impl RateLimiter {
    /// `permits_per_second` must be finite and greater than zero.
    /// `burst_capacity` must be at least 1.
    pub fn new(permits_per_second: f64, burst_capacity: u32) -> Result<Self, Error>;

    /// Tries to take one permit immediately.
    pub fn try_acquire(&self) -> bool;

    /// Waits until a permit becomes available.
    pub async fn acquire(&self) -> Result<(), Error>;

    /// Waits until a permit becomes available or the timeout expires.
    pub async fn acquire_timeout(&self, timeout: Duration) -> Result<(), Error>;
}

pub struct RateLimiterRegistry {
    // implementation details hidden
}

impl RateLimiterRegistry {
    pub fn new() -> Self;

    /// Registers or replaces a limiter under `name` and returns the stored limiter.
    pub fn register<S: Into<String>>(&self, name: S, limiter: RateLimiter) -> Arc<RateLimiter>;

    pub fn get<S: AsRef<str>>(&self, name: S) -> Option<Arc<RateLimiter>>;

    /// Atomically returns an existing limiter or creates, stores, and returns a new one.
    pub fn get_or_create<S, F>(&self, name: S, create_fn: F) -> Arc<RateLimiter>
    where
        S: Into<String>,
        F: FnOnce() -> RateLimiter;

    pub fn remove<S: AsRef<str>>(&self, name: S) -> Option<Arc<RateLimiter>>;
}
```

Semantics to Follow

1. The bucket holds up to `burst_capacity` permits.
2. New limiters start with a full bucket.
3. Permit refill is continuous over time, not only in whole-second jumps.
4. `try_acquire` must not wait.
5. `acquire_timeout` returns `Error::Timeout` if waiting takes too long.
6. `register` replaces an existing limiter of the same name.
7. `get_or_create` must behave atomically with respect to the registry map.

Hints

1. Use a mutex-protected internal state structure.
2. Do not hold a mutex guard across an `.await`.
3. `std::sync::Mutex` is fine here if the critical sections are short and non-awaiting.
4. Use Tokio time utilities in tests so you can write deterministic time-based tests.

Example Usage

Single Rate Limiter

```rust
use async_rate_limiter::RateLimiter;
use std::time::Duration;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let limiter = RateLimiter::new(5.0, 1)?;

    for i in 0..10 {
        limiter.acquire().await?;
        println!("operation {i}");
        tokio::time::sleep(Duration::from_millis(50)).await;
    }

    Ok(())
}
```

Registry Example

```rust
use async_rate_limiter::{RateLimiter, RateLimiterRegistry};
use std::sync::Arc;
use std::time::Duration;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    let registry = Arc::new(RateLimiterRegistry::new());

    registry.register("login", RateLimiter::new(5.0, 1)?);
    registry.register("public_api", RateLimiter::new(100.0, 20)?);

    let endpoint_limiter = registry.get("login").unwrap();
    endpoint_limiter.acquire_timeout(Duration::from_millis(100)).await?;

    Ok(())
}
```

What To Test

1. burst consumption: a fresh limiter should allow `burst_capacity` immediate acquisitions
2. refill over time: permits should become available again after enough simulated time passes
3. timeout behavior: waiting should fail when the timeout is too short
4. registry behavior: insert, replace, get, remove, $\text{get}_{\text{orcreate}}$

If you use `#[tokio::test(start_paused = true)]`, your tests become much easier to reason about.

Design Notes

This is a library-writing lab, so the API matters.

1. `try_acquire` is synchronous on purpose. If it does not wait, it should not be `async`.
2. `new` is fallible because invalid rate-limiter configuration is a normal input error, not a panic-worthy event.

3. `burst_capacity` is used instead of the older `max_burst` wording because the latter was ambiguous.
4. Combining multiple limiters in a single logical operation is subtle: if one acquisition succeeds and the next fails, you have partially consumed capacity. The simple example above does not solve that problem.

Skeleton

```rust
use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::time::Duration;
use tokio::time::{sleep, timeout, Instant};

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Error {
    Timeout,
    InvalidConfig,
}

pub struct RateLimiter {
    // TODO
}

struct RateLimiterState {
    // TODO
}

impl RateLimiter {
    pub fn new(permits_per_second: f64, burst_capacity: u32) -> Result<Self, Error> {
        todo!()
    }

    pub fn try_acquire(&self) -> bool {
        todo!()
    }

    pub async fn acquire(&self) -> Result<(), Error> {
        todo!()
    }

    pub async fn acquire_timeout(&self, timeout: Duration) -> Result<(), Error> {
        todo!()
    }
}

pub struct RateLimiterRegistry {
    // TODO
}

impl RateLimiterRegistry {
    pub fn new() -> Self {
        todo!()
    }
```

```
    pub fn register<S: Into<String>>(&self, name: S, limiter: RateLimiter) -> Arc<RateLimiter> {
        todo!()
    }

    pub fn get<S: AsRef<str>>(&self, name: S) -> Option<Arc<RateLimiter>> {
        todo!()
    }

    pub fn get_or_create<S, F>(&self, name: S, create_fn: F) -> Arc<RateLimiter>
    where
        S: Into<String>,
        F: FnOnce() -> RateLimiter,
    {
        todo!()
    }

    pub fn remove<S: AsRef<str>>(&self, name: S) -> Option<Arc<RateLimiter>> {
        todo!()
    }
}
```